

MERN FULLSTACK REPORT

TITLE : DocConnect – Doctor Appointment Platform

TEAM NAME : INFINITE INNOVATIONS

TEAM NUMBER : 07

Name of Student	Register Number
Mohamed Syed Ali	99230040673
Mitnala Srikanth	99230040672
Matam Vandana	99230040665
Nagireddy Meghana	99230040686
Modugapalam Shanmugam Reshma Reddy	99230040039
Manne Durga Bhavani	99230040660

Guide By

Ajusha

ABSTRACT

DOC CONNECT is a web-based doctor appointment and token booking management system developed using the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**. The main objective of this project is to make the doctor appointment process easier, faster, and more convenient for patients while helping doctors manage their appointments efficiently. The system reduces the need for manual appointment scheduling and provides a centralized platform for managing patient and doctor information.

DOC CONNECT provides three main user roles: **Patient, Doctor, and Admin**. Patients can create an account, log in securely, view available doctors based on their specializations, and book appointments online. During the booking process, patients can enter their personal and medical details, select a suitable appointment date, and choose an available time slot. After completing the booking process, patients can view their appointment details and booking status.

The system also includes a **payment module** that displays the appointment summary and a fixed consultation fee of **₹200**. A dummy QR code and simulated payment confirmation are provided to demonstrate the payment workflow. Actual online payment gateway integration is not included in the current version.

Doctors have a separate login facility and dashboard through which they can view their assigned appointments. They can access patient information, appointment dates, time slots, and symptoms, and update the appointment status. The backend uses authenticated APIs to retrieve appointments associated with the logged-in doctor. The contact module provides hospital information, including address, contact numbers, email details, working hours, and social media links.

The frontend is developed using **React.js**, while **Node.js and Express.js** handle backend operations and API communication. **MongoDB** is used to store patient, doctor, and appointment information. **Mongoose** manages database operations, **JWT** supports authentication, and **bcryptjs** is used for password hashing.

Overall, DOC CONNECT aims to provide a simple, organized, and user-friendly platform for online doctor appointment booking and management. It improves convenience for patients, supports doctors in managing their appointments, and provides a foundation for future enhancements such as real-time token tracking, online payment integration, appointment notifications, and video consultation facilities.

TABLE OF CONTENTS

TITLE		PAGE NO.
ABSTRACT		**ii**
LIST OF TABLES		**V**
LIST OF FIGURES		**V**
CHAPTER 1	**INTRODUCTION**	**1**
1.1	Introduction	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Scope of the Project	
CHAPTER 2	**SYSTEM ANALYSIS**	**4**
2.1	Introduction	
2.2	Analysis of the Existing System	
2.3	Limitations of the Existing System	
2.4	Proposed System	
2.5	Features of the Proposed System	
2.6	Functional Requirements	
2.7	Non-Functional Requirements	
2.8	Hardware Requirements	
2.9	Software Requirements	
2.10	Feasibility Study	
2.11	Summary	
CHAPTER 3	**SYSTEM DESIGN**	**11**
3.1	Introduction	
3.2	System Architecture	
3.3	Data Flow Design	
3.4	Database Design	
3.5	Module Design	
3.6	Security Design	
3.7	Summary	
CHAPTER 4	**SYSTEM IMPLEMENTATION**	**14**

4.1	Introduction	
4.2	Frontend Implementation	
4.3	Backend Implementation	
4.4	Database Implementation	
4.5	Authentication Implementation	
4.6	Appointment Booking Implementation	
4.7	Payment Implementation	
4.8	Doctor Dashboard Implementation	
4.9	Summary	
CHAPTER 5	**SYSTEM TESTING**	**16**
5.1	Introduction	
5.2	Types of Testing	
5.3	Test Cases	
5.4	Testing Result	
5.5	Summary	
CHAPTER 6	**RESULTS AND SCREENSHOTS**	**18**
6.1	Introduction	
6.2	Patient Module Results	
6.3	Doctor Module Results	
6.4	Contact Module Results	
6.5	Overall System Result	
CHAPTER 7	**CONCLUSION AND FUTURE ENHANCEMENT**	**23**
7.1	Conclusion	
7.2	Future Enhancements	
7.3	Final Summary	

LIST OF TABLES

TABLES	DETAILS	PAGE NO.
Table 1	Hardware requirements	9
Table 2	Technical Feasibility	9
Table 3	Test cases	16

LIST OF FIGURES

FIGURES	DETAILS	PAGE NO.
Figure 1	System Architecture	11
Figure 2	Home Page	18
Figure 3	Patient Home Page	18
Figure 4	Doctor Selection Page	19
Figure 5	Date and Time Selection Page	19
Figure 6	Payment Page	19
Figure 7	Booking Status Page	20
Figure 8	Doctor Login Page	20
Figure 9	Doctor Dashboard	21
Figure 10	Doctor Appointments Page	21
Figure 11	Manage Schedule	21
Figure 12	Contact Page	22
Figure 13	Admin dashboard	22

CHAPTER 1

INTRODUCTION

1.1 Introduction

DOC-CONNECT is a web-based doctor appointment and token booking management system developed to simplify the process of scheduling medical appointments between patients and doctors. The system provides a digital platform where patients can register, log in, select doctors, choose appointment dates and time slots, and complete the booking process.

Traditional hospital appointment systems often involve manual registration, long waiting times, and difficulties in managing appointments. Patients may need to visit hospitals directly or contact reception staff to schedule consultations. These processes can consume time and create difficulties for both patients and hospital administrators.

To overcome these challenges, DOC-CONNECT provides an online platform that allows patients to book appointments conveniently. The system includes separate portals for patients, doctors, and administrators, with role-based access to their respective functionalities.

Patients can manage their bookings and view appointment information. Doctors can log in to their dashboard to view appointments assigned to them and manage appointment statuses. Administrators can manage the overall system, including doctors, patients, and appointments.

The application is developed using the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js. JWT authentication is used to secure access to the system, while MongoDB stores the application data.

The main objective of DOC-CONNECT is to provide a simple, secure, and efficient digital appointment management solution that reduces manual work and improves coordination between patients, doctors, and hospital administrators.

1.2 Problem Statement

In many healthcare institutions, appointment scheduling and patient management are still handled through manual procedures or basic systems. These methods can lead to several problems, including:

- Patients spending considerable time waiting to book appointments.
- Difficulty in identifying available doctors and appointment slots.
- Manual errors during appointment registration.
- Challenges in maintaining patient and appointment records.
- Difficulty for doctors in accessing their assigned appointments.
- Lack of centralized monitoring and management by administrators.

These problems can affect the efficiency of hospital operations and create inconvenience for patients and medical staff.

Therefore, a centralized web-based system is required to simplify appointment booking, organize patient information, and provide separate management facilities for patients, doctors, and administrators.

DOC-CONNECT is developed to address these issues through an integrated online appointment and token booking platform.

1.3 Objectives of the Project

The primary objective of the DOC-CONNECT project is to develop a web-based platform that simplifies doctor appointment booking and management.

The specific objectives are:

1. To develop a user-friendly patient registration and login system.
2. To provide a separate and secure login facility for doctors and administrators.
3. To allow patients to view and select available doctors based on their specializations.
4. To provide date and time-slot selection for appointment booking.
5. To implement a payment interface with a fixed consultation fee.
6. To allow patients to view their booking details and appointment status.
7. To provide doctors with a dashboard to view their assigned appointments.
8. To ensure that doctors can access only the appointments assigned to them.
9. To provide administrators with centralized system management facilities.
10. To store and manage patient, doctor, and appointment information using MongoDB.

1.4 Scope of the Project

The scope of the DOC-CONNECT system is to provide an online platform for managing doctor appointments and patient information within a healthcare environment.

The system includes the following major functionalities:

Patient Module

- Patient registration and login.
- Viewing available doctors and their specializations.
- Entering personal and medical details for appointments.
- Selecting appointment dates and time slots.
- Accessing the payment interface.
- Viewing booking information and appointment status.

Doctor Module

- Secure doctor login.

- Individual doctor dashboard.
- Viewing assigned patient appointments.
- Accessing patient details and symptoms.
- Managing appointment status.

Admin Module

- Centralized administration of the system.
- Managing doctor and patient records.
- Monitoring appointments.
- Managing system activities and reports.

Payment Module

The system provides a payment interface displaying the consultation fee and QR code. Currently, the payment process is simulated and does not involve a live payment gateway.

Database Management

MongoDB is used to maintain patient details, doctor information, authentication records, and appointment data.

The project focuses on improving appointment management, reducing manual work, and providing convenient access to healthcare appointment services.

CHAPTER 2

SYSTEM ANALYSIS

This chapter explains the requirements, resources, technologies, and overall functionality of the DOC-CONNECT system. It describes how the system operates, what facilities it provides to different users, and the requirements needed to develop and run the application.

The content below is based on your actual MERN stack project.

2.1 Introduction

System analysis is an important phase in software development that helps identify the requirements, functionalities, and resources needed to develop a software application.

The DOC-CONNECT system is designed to simplify the process of doctor appointment booking and management. It provides a centralized web-based platform that connects patients, doctors, and administrators through separate interfaces.

The system allows patients to register, log in, select doctors, choose appointment dates and time slots, and proceed through the payment interface. Doctors can access their dashboards to view appointments assigned to them and manage appointment statuses. The administrator is provided with a centralized interface for managing and monitoring system activities.

The system analysis phase identifies the functional and non-functional requirements, hardware requirements, software requirements, and feasibility of the proposed application.

2.2 Analysis of the Existing System

In the traditional appointment management system, patients generally need to visit hospitals or contact reception staff to schedule appointments with doctors. Appointment details and patient information may be maintained manually or through separate record management systems.

This approach can create difficulties in managing appointments, maintaining patient records, and coordinating between doctors and hospital administration.

The existing approach may involve the following activities:

- Patients contact the hospital to enquire about doctor availability.
- Appointment details are recorded manually.
- Doctors receive appointment information through hospital staff.
- Patient records are maintained separately.
- Appointment status updates may require manual communication.
- Hospital administrators manage records and appointments through manual procedures.

These processes can increase administrative workload and make appointment management less convenient.

2.3 Limitations of the Existing System

The traditional appointment management approach has several limitations:

1. Time Consumption: Patients may spend considerable time visiting hospitals or contacting reception staff to schedule appointments.

2. Manual Record Management: Maintaining patient and appointment information manually can lead to errors and difficulties in retrieving records.

3. Appointment Management Difficulties: Managing multiple doctors and their appointments can become complicated.

4. Lack of Centralized Information: Patient, doctor, and appointment information may be maintained in separate records.

5. Communication Difficulties: Patients and doctors may face difficulties receiving timely appointment updates.

6. Limited Accessibility: Patients may need to contact the hospital directly to obtain appointment information.

7. Administrative Workload: Hospital staff may need to perform several repetitive tasks related to appointment registration and record maintenance.

2.4 Proposed System

The proposed DOC-CONNECT system is a web-based doctor appointment and token booking management application developed using the MERN stack.

It provides separate login facilities and dashboards for patients, doctors, and administrators. The system simplifies appointment scheduling by allowing patients to select doctors, choose dates and time slots, and access the payment interface through an online platform.

The application uses MongoDB to store patient, doctor, and booking information. Express.js and Node.js handle backend operations, while React.js provides an interactive frontend interface.

JWT authentication is implemented to secure user access. The booking system associates each appointment with the corresponding doctor's MongoDB ID, allowing doctors to view only their assigned appointments.

The proposed system aims to reduce manual work, improve appointment organization, and provide a convenient digital environment for managing healthcare appointments.

2.5 Features of the Proposed System

The DOC-CONNECT system provides the following major features:

1. Patient Management

- Patient registration and login.
- Patient dashboard.
- Viewing available doctors.

- Entering patient details during booking.
- Viewing appointment information.

2. Doctor Management

- Separate doctor login.
- Doctor dashboard.
- Viewing assigned appointments.
- Accessing patient details and symptoms.
- Managing appointment statuses.

3. Appointment Booking

- Doctor selection based on specialization.
- Appointment date selection.
- Prevention of selecting past dates.
- Time-slot selection.
- Storage of booking information in MongoDB.

4. Payment Interface

- Display of booking summary.
- Fixed consultation fee of ₹200.
- QR code display.
- Simulated payment confirmation.
- Navigation to the appointment status page.

5. Admin Management

- Centralized dashboard for managing the application.
- Doctor and patient record management.
- Monitoring appointment information.
- Managing overall system activities.

6. Authentication and Security

- Separate login for different user roles.
- Password hashing using bcryptjs.
- JWT-based authentication.
- Role-based access restrictions.

- Doctor-specific appointment access.

2.6 Functional Requirements

Functional requirements describe the operations and services that the system must provide to its users.

The functional requirements of DOC-CONNECT are classified according to the three user roles.

2.6.1 Patient Functional Requirements

- The system must allow new patients to register using their name, email, password, and phone number.
- Registered patients must be able to log in using valid credentials.
- The system must redirect authenticated patients to the home page.
- Patients must be able to view the list of available doctors and their specializations.
- Patients must be able to enter appointment-related details such as age, gender, weight, and symptoms.
- Patients must be able to select an appointment date and time slot.
- The system must prevent patients from selecting past dates.
- Patients must be able to access the payment interface.
- Patients must be able to view their booking details and appointment status.

2.6.2 Doctor Functional Requirements

- The system must provide a separate login facility for doctors.
- Doctors must be redirected to their individual dashboard after successful authentication.
- Doctors must be able to view their scheduled appointments.
- Doctors must be able to view patient names, appointment dates, time slots, and symptoms.
- Doctors must be able to update appointment statuses.
- Each doctor must be able to access only their assigned patient bookings.
- Doctors must be able to log out securely.

2.6.3 Admin Functional Requirements

- The system must provide a separate administrator login.
- The administrator must be able to access the admin dashboard.
- The administrator must be able to view doctor and patient information.

- The administrator must be able to manage doctor records.
- The administrator must be able to monitor appointment information.
- The administrator must be able to manage overall system activities.

Note: Include the admin operations in the final report according to the features actually completed in your project.

2.7 Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance expectations of the application.

The following non-functional requirements are considered for DOC-CONNECT.

1. Security

The system uses JWT authentication to protect backend routes and bcryptjs to securely hash user passwords. Access to information is restricted according to user roles.

2. Usability

The application provides a simple and user-friendly interface so that patients, doctors, and administrators can easily navigate their respective dashboards.

3. Performance

The system should process user requests, retrieve appointment information, and display dashboard data efficiently.

4. Reliability

The application should maintain accurate patient, doctor, and booking information in the MongoDB database.

5. Scalability

The system should support future improvements such as additional doctors, more patients, and additional appointment management features.

6. Maintainability

The application is developed using separate frontend components, backend routes, and database models, making it easier to maintain and update individual modules.

7. Accessibility

The web application is designed with responsive layouts to support access through different screen sizes and devices.

2.8 Hardware Requirements

The following hardware requirements are suitable for developing and running the DOC-CONNECT application.

Component	Minimum requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	10 GB free space
Display	Standard monitor
Input Devices	Keyboard and mouse
Network	Internet connection

Table 1 Hardware requirements

2.9 Software Requirements

The following software and technologies are used in the development of DOC-CONNECT.

2.10 Feasibility Study

A feasibility study is conducted to determine whether the proposed system can be developed and operated successfully.

The DOC-CONNECT project is evaluated based on the following feasibility factors.

2.10.1 Technical Feasibility

The project is technically feasible because it uses widely available technologies such as React.js, Node.js, Express.js, and MongoDB.

These technologies provide the necessary facilities for developing interactive web pages, backend APIs, database operations, and secure authentication.

The MERN stack also supports modular development and future enhancements.

Software / Technology	Purpose
React.js	Frontend development
Node.js	Backend runtime environment
Express.js	Backend API development
MongoDB	Database management
Mongoose	MongoDB object modelling
JavaScript	Application programming
HTML5	Web page structure
CSS3	Styling and responsive design
JWT	User authentication
bcryptjs	Password hashing
VS Code	Development environment
Postman	API testing

Table 2 Technical Feasibility

2.10.2 Economic Feasibility

The project is economically feasible because the major development technologies used are open-source and freely available.

Visual Studio Code, Node.js, Express.js, React.js, and MongoDB Community Edition can be used without purchasing expensive development software.

Therefore, the development cost can be minimized.

2.10.3 Operational Feasibility

The system is operationally feasible because it provides a simple interface for patients, doctors, and administrators.

Patients can book appointments online without depending entirely on manual registration procedures. Doctors can view their assigned appointments through their dashboards, while administrators can manage the system through a centralized interface.

The application requires basic computer or smartphone knowledge to operate.

2.10.4 Schedule Feasibility

The project is developed using modular implementation, where individual modules such as authentication, patient management, doctor management, booking, and payment are developed separately.

This approach helps organize development activities and makes it easier to complete and test the project within the available time.

2.11 Summary

This chapter discussed the system analysis of the DOC-TOKEN application, including the existing system, its limitations, the proposed system, functional requirements, non-functional requirements, hardware and software requirements, and feasibility study.

The analysis identifies the essential requirements needed to develop an efficient doctor appointment and token booking management system. It also establishes the foundation for the system design and implementation phases discussed in the following chapters.

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

System design explains the structure, modules, database, and workflow of the DOC-TOKEN project. It describes how patients, doctors, and administrators interact with the application.

3.2 System Architecture

DOC-TOKEN follows the MERN Stack Architecture.

- Frontend: React.js – provides the user interface.
- Backend: Node.js and Express.js – handles requests and business logic.
- Database: MongoDB – stores user and appointment details.
- Authentication: JWT – manages secure user login.
- Password Security: bcryptjs – hashes passwords.

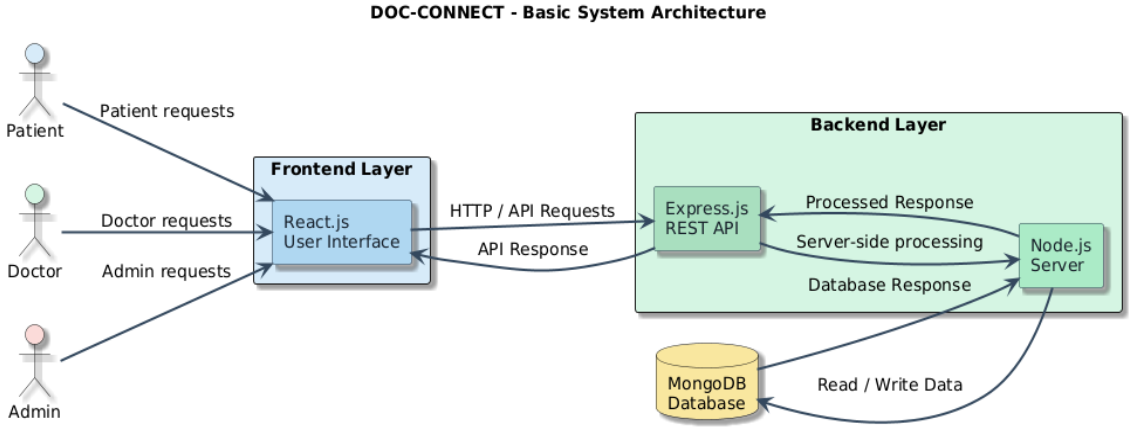


Fig 1 System Architecture

3.3 Data Flow Design

The DOC-CONNECT system acts as the central process connecting patients, doctors, and administrators.

Level 0

- Patient sends registration and booking details.
- Doctor requests assigned appointment details and updates status.
- Admin manages system information through the admin module.

Level 1 – Data Flow

1. User registers or logs in.
2. The system verifies user credentials.

3. Patient selects a doctor, date, and time slot.
4. Patient completes the simulated payment process.
5. Booking details are stored in MongoDB.
6. Doctor views assigned appointments and updates their status.
7. Patient views booking information and appointment status.

3.4 Database Design

The project uses MongoDB to store application data. Mongoose is used to define and manage database models.

Main collections include:

- Users: Stores patient login and profile details.
- Doctors: Stores doctor names, specializations, and login details.
- Bookings: Stores patient ID, doctor ID, appointment date, time, age, gender, weight, symptoms, and booking status.

3.5 Module Design

The system is divided into the following modules:

1. Patient Module

- Registration and login.
- Patient dashboard.
- Doctor selection.
- Appointment booking.
- Booking status.

2. Doctor Module

- Doctor login.
- Doctor dashboard.
- View assigned appointments.
- Update appointment status.

3. Booking Module

- Manages appointment details.
- Connects patients with selected doctors.
- Stores appointment information in MongoDB.

4. Payment Module

- Displays booking summary.
- Shows consultation fee of ₹200.
- Provides simulated QR payment confirmation.

5. Admin Module

- Centralized system management.
- Doctor and patient management.
- Appointment monitoring.

3.6 Security Design

The system uses the following security features:

- JWT Authentication: Verifies authenticated users.
- bcryptjs: Securely hashes passwords.
- Role-Based Access: Restricts access according to user roles.
- Doctor-Specific Access: Ensures doctors can view only their assigned appointments.
- Protected APIs: Restricts unauthorized access to backend resources.

3.7 Summary

This chapter explained the system architecture, data flow, database structure, modules, and security design of DOC-CONNECT. These design components provide a structured foundation for developing and maintaining the application

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Introduction

System implementation is the process of developing the DOC-CONNECT application using the MERN stack. The system provides separate interfaces for patients and doctors to manage appointment bookings.

4.2 Frontend Implementation

The frontend is developed using **React.js, HTML, CSS, and JavaScript**.

- Patient registration and login pages.
- Patient dashboard and doctor selection.
- Date and time-slot selection.
- Payment and booking status pages.
- Doctor login and dashboard.
- Contact page with hospital information.

4.3 Backend Implementation

The backend is developed using **Node.js and Express.js**.

- Handles patient and doctor authentication.
- Processes appointment booking requests.
- Retrieves doctor-specific appointments.
- Allows doctors to update appointment status.
- Provides APIs for communication between frontend and database.

4.4 Database Implementation

MongoDB is used to store application data, with **Mongoose** for database operations.

The main data includes:

- Patient details and login information.
- Doctor details and specializations.
- Booking details, including patient ID, doctor ID, date, time, and symptoms.
- Appointment status.

4.5 Authentication Implementation

The system uses **JWT (JSON Web Token)** for user authentication and **bcryptjs** for password hashing.

- Provides secure login.
- Protects restricted routes.
- Supports role-based access.
- Ensures doctors can view their assigned appointments.

4.6 Appointment Booking Implementation

The booking process allows patients to:

1. Select a doctor.
2. Choose an appointment date.
3. Select an available time slot.
4. Enter appointment details.
5. Complete the simulated payment step.
6. View the booking status.

The booking information is stored in **MongoDB** and linked to the selected doctor and patient.

4.7 Payment Implementation

The payment page displays the booking summary and a fixed consultation fee of **₹200**. It includes a dummy QR code and a simulated payment confirmation process.

Note: Actual online payment processing is not integrated into the current system.

4.8 Doctor Dashboard Implementation

The doctor dashboard provides:

- Welcome page and dashboard navigation.
- Appointment details.
- Patient information and symptoms.
- Appointment date and time.
- Status updates for assigned appointments.

4.9 Summary

The DOC-CONNECT system is implemented using the **MERN stack**. It connects the frontend, backend, and MongoDB database to support patient registration, appointment booking, simulated payment, and doctor appointment management.

CHAPTER 5

SYSTEM TESTING

5.1 Introduction

System testing is performed to check whether the DOC-TOKEN application works correctly. It helps identify errors and ensures that the main features function as expected.

5.2 Types of Testing

- **Unit Testing:** Checks individual components and functions.
- **Integration Testing:** Checks communication between the frontend, backend, and database.
- **Functional Testing:** Verifies that application features work according to requirements.
- **Validation Testing:** Checks whether the entered details are accepted correctly.
- **User Interface Testing:** Checks the layout, navigation, and responsiveness of the application.

5.3 Test Cases

Test Case	Expected Result
Patient registration	Patient account should be created successfully.
Patient login	Valid credentials should allow login.
Invalid login	Error message should be displayed.
Doctor login	Doctor should access the doctor dashboard.
Doctor selection	Selected doctor details should be displayed.
Date selection	Past dates should not be selectable.
Time-slot selection	Selected time slot should be displayed.
Appointment booking	Booking details should be saved in MongoDB.
Payment simulation	Payment confirmation message should be displayed.
Booking status	Patient should be able to view booking status.
Doctor appointments	Doctor should view assigned appointments.
Appointment status update	Doctor should be able to update appointment status.

Table 3 Test Cases

5.4 Testing Result

The test cases are designed to verify the major functions of the DOC-CONNECT application, including **registration, login, appointment booking, simulated payment, and doctor appointment management**.

5.5 Summary

System testing helps verify the functionality and reliability of the DOC-CONNECT application. It ensures that the main modules work together and support the appointment booking process.

CHAPTER 6

RESULTS AND SCREENSHOTS

6.1 Introduction

This chapter presents the **results of the DOC-CONNECT application**. The developed system provides separate modules for patients and doctors to manage the online appointment booking process.

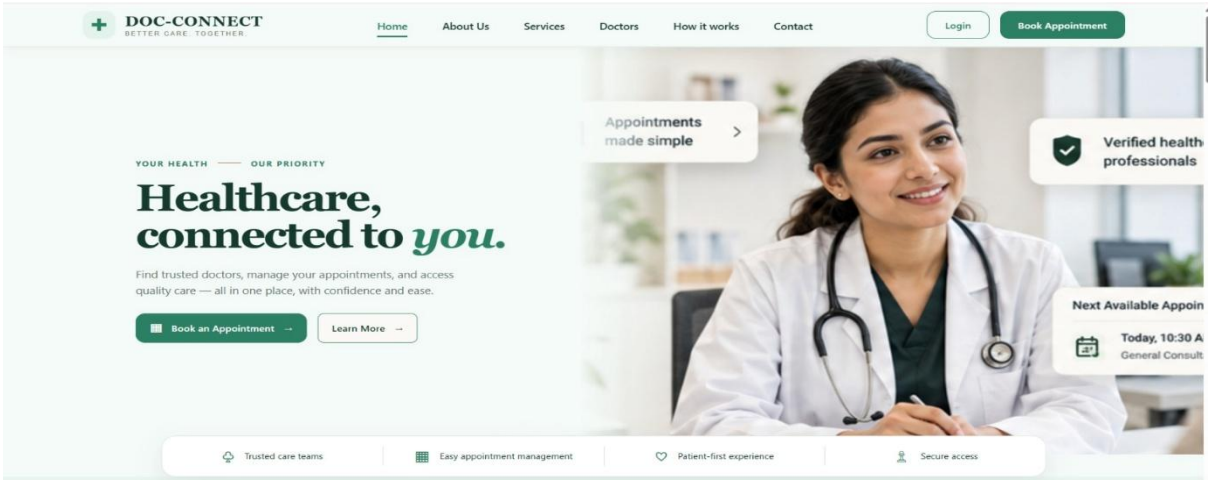


Fig 2: Home Page

6.2 Patient Module Results

- Patients can register and log in.
- Patients can view the available doctors.
- Patients can select an appointment date and time slot.
- Patients can complete the simulated payment process.
- Patients can view their booking status.

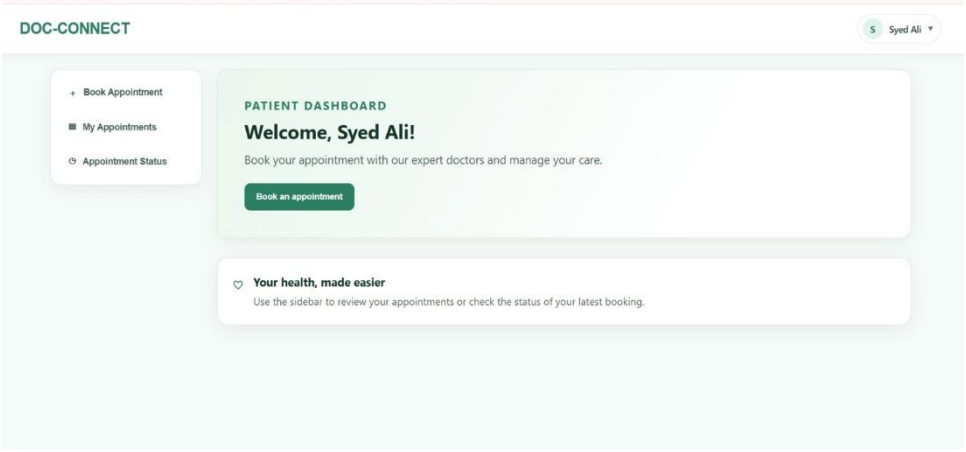


Fig 3 : Patient Home Page

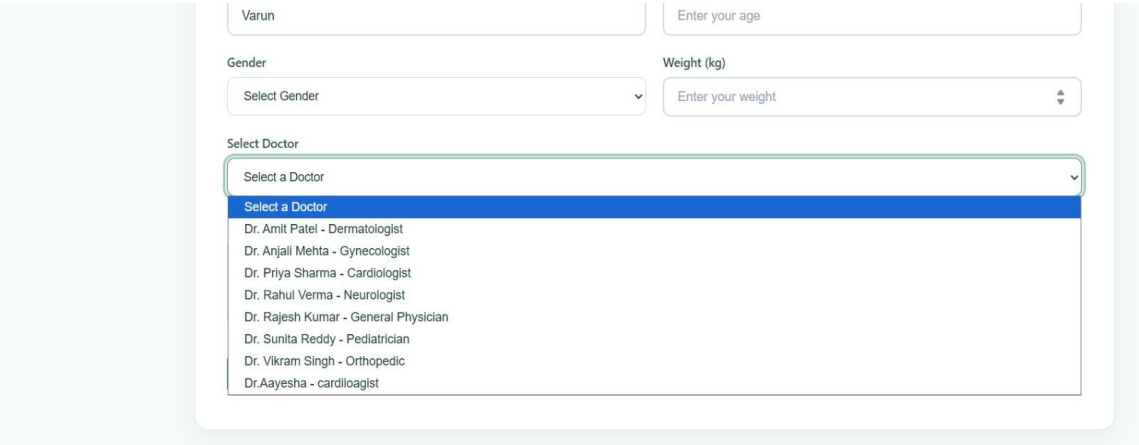


Fig 4: Doctor Selection Page

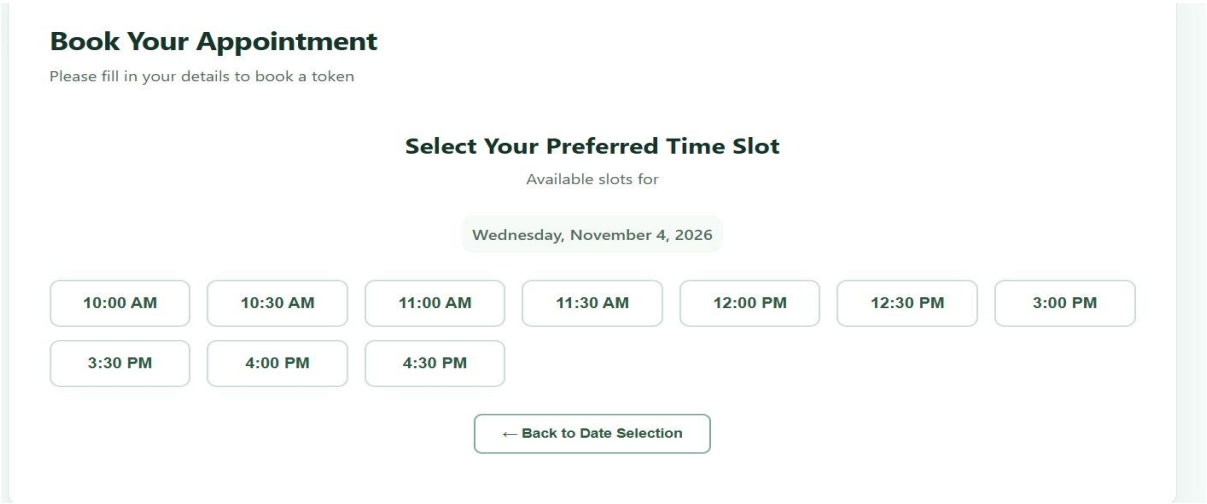


Fig 5: Date and Time Selection Page

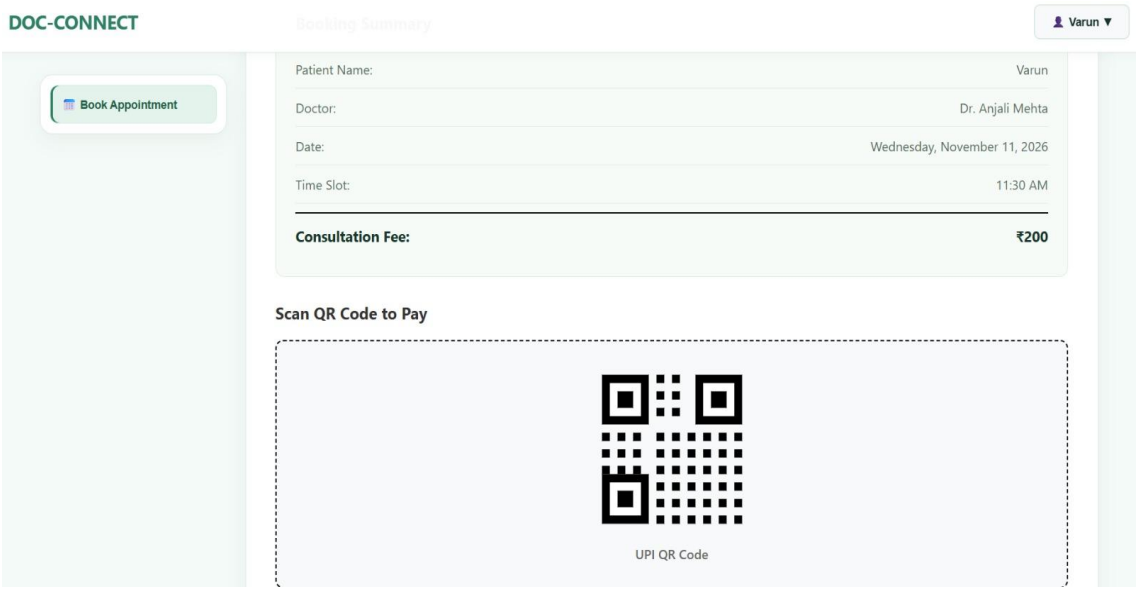


Fig 6: Payment Page

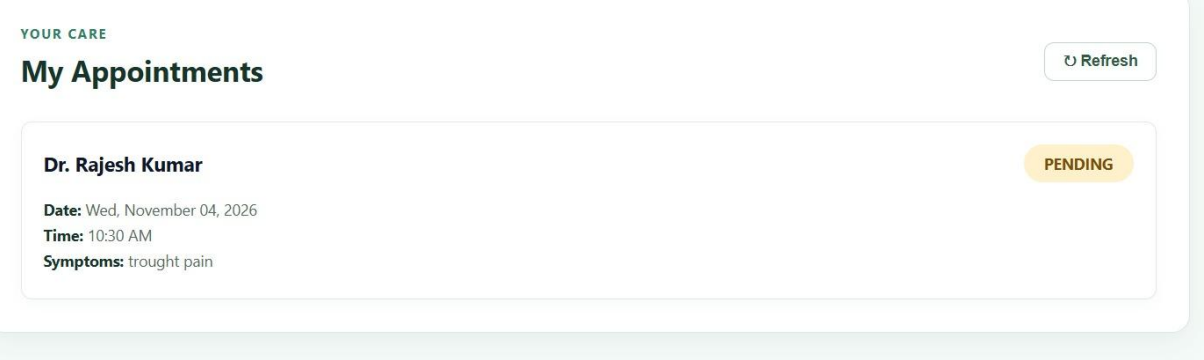


Fig 7: Booking Status Page

6.3 Doctor Module Results

- Doctors can log in to their dashboard.
- Doctors can view their assigned appointments.
- Patient details, symptoms, date, and time are displayed.
- Doctors can update appointment

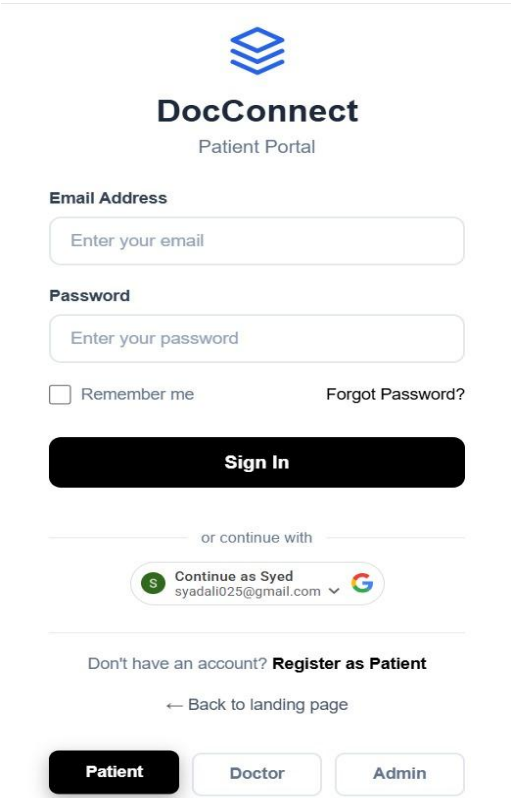


Fig 8: Doctor Login Page

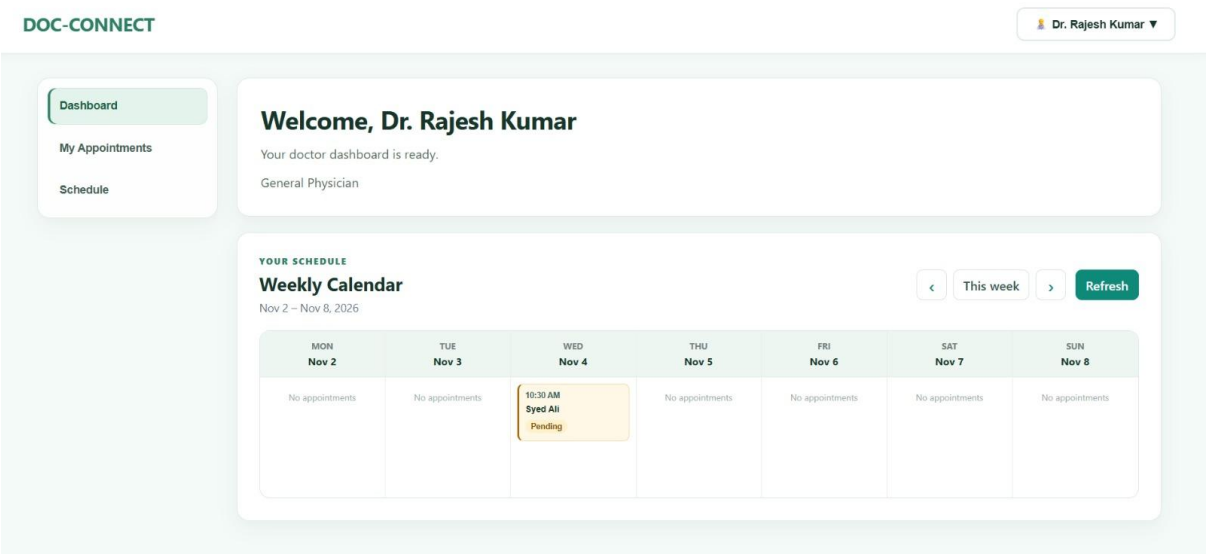


Fig 9: Doctor Dashboard

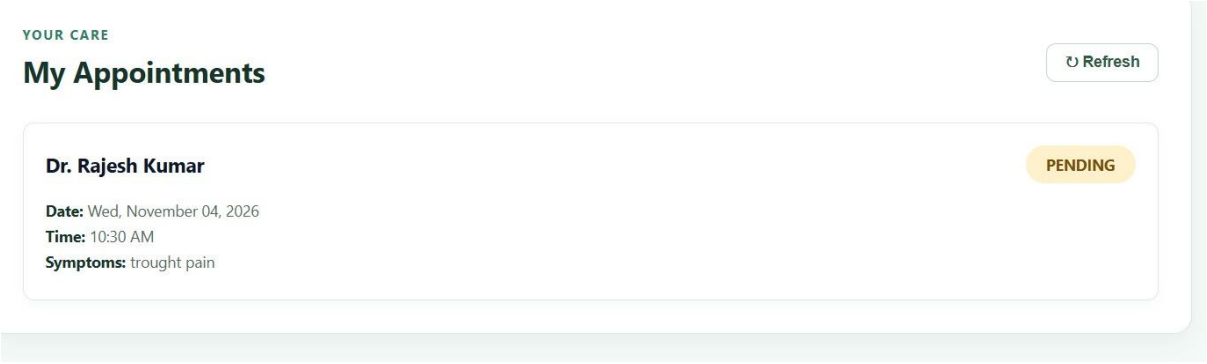


Fig 10: Doctor Appointments Page

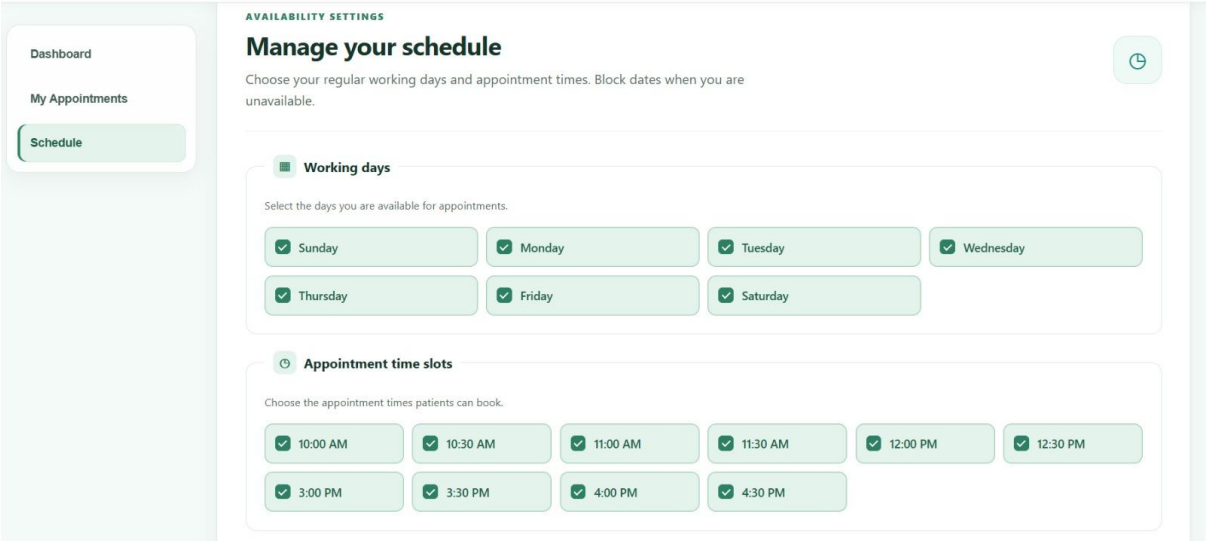


Fig 11 : Manage Schedule

6.4 Contact Module Results

The contact page displays hospital details, phone numbers, email addresses, working hours, social media links, and a map placeholder.

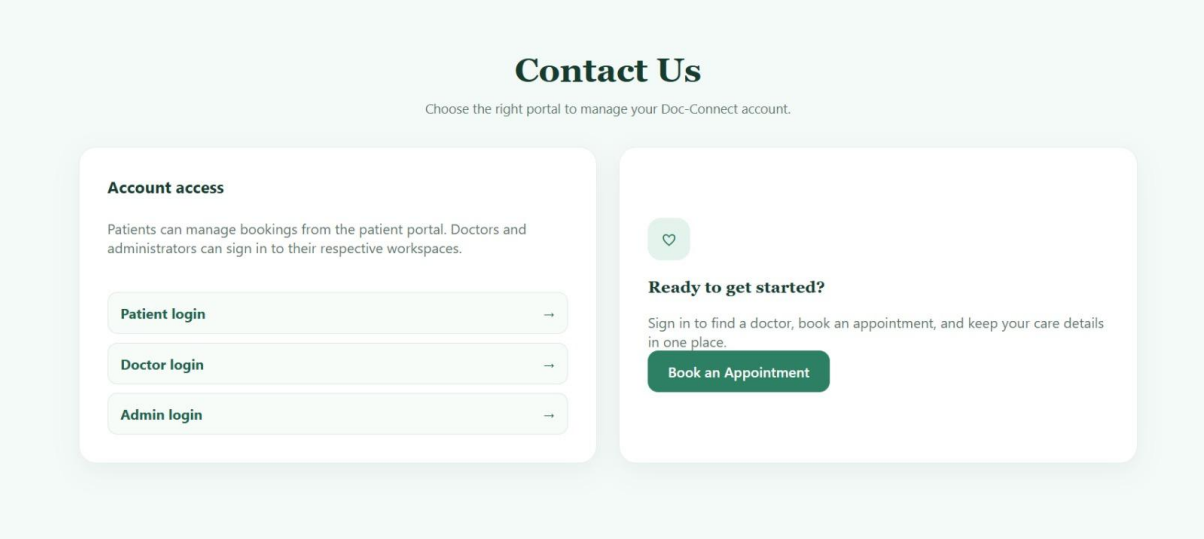


Fig 12: Contact Page

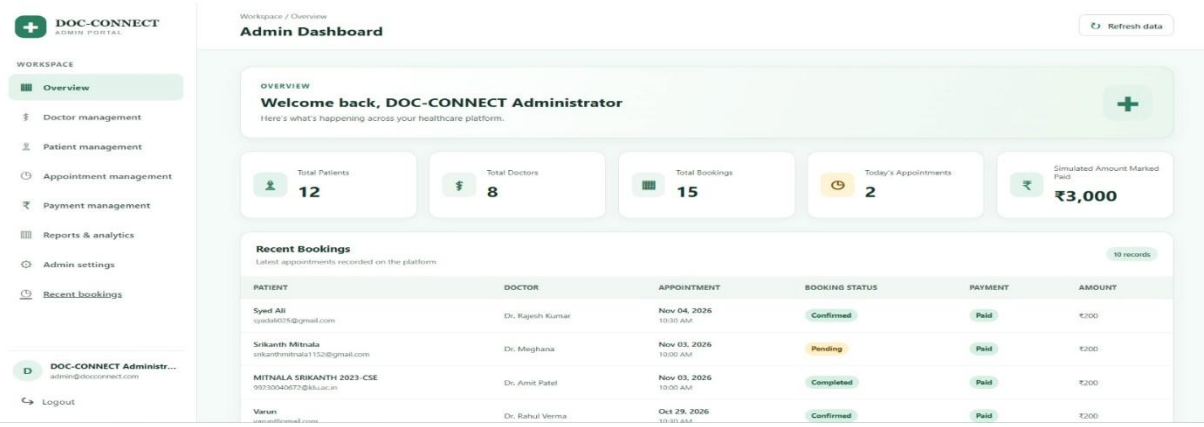


Fig 13 : Admin dashboard

6.5 Overall System Result

The DOC-CONNECT application provides an organized platform for **patient registration, doctor selection, appointment booking, simulated payment, and doctor appointment management**. The system stores booking information in MongoDB.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

The **DOC-CONNECT** project was developed to simplify the process of doctor appointment booking through an online platform. It uses the **MERN stack (MongoDB, Express.js, React.js, and Node.js)** to provide an organized and user-friendly application.

The system allows patients to register, select doctors, choose appointment dates and time slots, and view booking status. Doctors can log in, view their assigned appointments, and update appointment status.

The project reduces the need for manual appointment management and provides a convenient way to manage patient bookings. The payment process is currently simulated and does not use a real payment gateway.

7.2 Future Enhancements

The following features can be added in future versions:

- **Online Payment Gateway:** Integrate secure payment services for real transactions.
- **SMS and Email Notifications:** Send booking confirmations and appointment reminders.
- **Admin Dashboard:** Provide complete management of doctors, patients, and appointments.
- **Live Token Tracking:** Allow patients to track their token position in real time.
- **Video Consultation:** Add online consultation facilities.
- **Mobile Application:** Develop an Android and iOS application.
- **Reports and Analytics:** Generate detailed appointment and payment reports.

7.3 Final Summary

DOC-CONNECT provides a digital solution for doctor appointment and token booking management. It demonstrates how web technologies can make the appointment process **simpler, faster, and more organized**. Future enhancements can make the system more efficient and useful for patients, doctors, and administrators.